

CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 03 · LECTURE NOTES

Time, Clocks, and Ordering

There is no now.

MODULE OUTCOMES

Explain why physical clocks cannot be trusted.

Define the happens-before relation.

Implement Lamport and vector clocks and state what each captures.

Explain hybrid logical clocks and TrueTime.

Explain why timestamps are the wrong conflict resolution.

1 Physical time lies

- **Clocks drift.** An ordinary quartz crystal is off by seconds per day, and its rate changes with temperature — so two machines in the same rack drift apart at different speeds.
- **NTP steps them backwards.** When a correction is applied, two successive readings of the wall clock can *decrease*, so a measured duration can come out negative. Code that subtracts two wall-clock readings has a latent bug.
- **Leap seconds.** A second is repeated or 'smeared' across a day, and **different vendors smear differently on the same day** — so two correctly configured machines genuinely disagree about what time it is.
- **Virtual machines pause.** A VM can lose seconds to live migration or to a host under memory pressure, and resumes with the wall clock having moved on without it.
- **And garbage collection pauses** (CSCE 605 Module 12). The process stops; the clock does not. **Code resumes believing very little time has passed and finds itself in the future** — which has caused real outages where a node held a lease it had already lost. **Use a monotonic clock for durations and the wall clock only for displaying times to humans**; mixing them is among the most common bugs in this subject.

A CLOCK READING IS A RANGE, NOT A POINT

Even when well synchronised, a clock reading carries uncertainty. NTP over the public internet gives tens of milliseconds of error; within a datacentre with good peering, perhaps one; PTP with hardware support, microseconds.

So 'this happened at 10:00:00.000' really means 'somewhere in a window around then', and comparing two such readings taken on different machines is meaningless whenever the difference between them is smaller than the combined uncertainty.

Most systems ignore this entirely and compare timestamps as though they were exact — which silently produces wrong orderings exactly in the close cases, which are the cases where the ordering mattered.

Google's Spanner does the opposite. Its TrueTime API returns an explicit interval rather than a point, and **a transaction waits out the uncertainty before committing** so that its timestamp is guaranteed to be in the past for every other node. **Correctness bought with latency** — typically a few milliseconds of commit wait — which is an honest and unusually explicit trade, and it requires GPS receivers and atomic clocks in every datacentre to keep the interval small.

2 Logical time

$a \rightarrow b$ (happens-before)

defined by three rules: **a and b are in the same process and a comes first; a is a send and b is the matching receive;** or **transitivity** through some intermediate c. **If neither $a \rightarrow b$ nor $b \rightarrow a$, the two events are concurrent.** **Concurrent does not mean simultaneous** — it means *neither could possibly have influenced the other*, which is the only thing a distributed system can actually determine. That reframing is the whole idea: give up on knowing what time it was, and settle for knowing what could have caused what.

```
LAMPORT CLOCK -- one integer per process
  local event:  c = c + 1
  send:         c = c + 1; attach c
  receive(t):   c = max(c, t) + 1

GUARANTEE: a -> b implies C(a) < C(b)
NOT TRUE:  C(a) < C(b) implies a -> b    <-- the catch
Gives a total order; CANNOT detect concurrency.

VECTOR CLOCK -- one integer per process, kept at every process
  local event:  V[self] += 1
  send:         V[self] += 1; attach all of V
  receive(w):   V = elementwise max(V, w); V[self] += 1

V(a) <= V(b) and strictly less somewhere <=> a -> b
neither <= the other                      <=> CONCURRENT

Detects concurrency EXACTLY. Costs O(n) per message.
```

The 'NOT TRUE' line is the one that gets missed, and missing it leads directly to misusing Lamport clocks for conflict detection. A Lamport clock tells you that if a caused b then a's timestamp is smaller — it does not tell you that a smaller timestamp implies causation, so two unrelated events get ordered arbitrarily and you cannot tell that from the numbers.

3 The clock choices

Clock	What it captures	What it costs
Physical (NTP)	Approximately real time, and no causality at all.	Free, and wrong at fine granularity — §1.
Lamport	A consistent total order over all events.	One integer. Cannot distinguish concurrent from ordered.
Vector	Causality, exactly.	O(n) space and bandwidth per message , which becomes prohibitive with many participants and requires handling membership changes.
Hybrid logical clock (HLC)	Causality, and stays within a bounded distance of real time.	A physical component plus a small logical counter. The practical default — it respects happens-before, is meaningful to humans, and fits in a fixed number of bytes. Used by CockroachDB, MongoDB, and others.
TrueTime	Real time with an explicit, bounded error interval.	GPS receivers and atomic clocks in every datacentre, plus commit wait. Spanner. Correct, and only available if you own the hardware.
Version vectors	Causality over <i>replicas</i> rather than processes.	O(replicas), which is far smaller than O(clients). Used by Dynamo-style stores and by CRDTs (Module 08).

4 Last write wins

'LAST WRITE WINS' DECIDES BY A CLOCK THAT IS WRONG

Two replicas accept concurrent writes to the same key; when they reconcile, keep whichever carries the later timestamp. It is simple, it is stateless, and it is extremely widespread — Cassandra's default, among many others.

But the two timestamps came from different machines with unsynchronised clocks (§1). If one node's clock is 200 ms fast, its writes always win — including the ones that genuinely happened first.

So data is discarded silently. There is no error returned, no log entry, no metric, and **no way to recover the lost write afterwards**, because the losing value was never stored anywhere. The failure is invisible by construction.

The alternatives, in increasing order of cost: detect the conflict with version vectors and surface it to the application or the user (Dynamo's sibling values); use a CRDT that merges without having to choose (Module 08 §3); or serialise the writes through consensus so concurrency never arises (Module 06).

Last-write-wins is acceptable only when silently losing a write is genuinely acceptable — for a cache or a presence indicator it may well be — and that should be a stated decision rather than a default inherited from a configuration file.

WHERE THIS MATTERS IN A GAME

Clients fundamentally disagree about when things happened, because each one observes the world through a different and varying delay. There is no shared present moment to appeal to.

So the server defines time. A single authoritative *tick number*, incremented at a fixed rate, replaces wall-clock time entirely — which is to say **it is a Lamport clock with a designated owner**, and the designation is what removes the ambiguity that a general Lamport clock leaves.

Clients estimate their offset from server time — by measuring round-trip time and smoothing it — and stamp each input with the server tick they believe they are acting on. The server then knows not just what the client did but *when the client thought it was doing it*.

And lag compensation rewinds the world to the tick the shooter actually saw (Module 11 §2) before testing the hit. **That is only possible because there is one authoritative timeline to rewind along** — which is a good illustration that choosing a clock model is an architectural decision, not an implementation detail.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Lamport — Time, Clocks, and the Ordering of Events in a Distributed System (free)	The paper that created §2. Short, readable, and one of the most cited papers in computing for good reason.
Kleppmann — Designing Data-Intensive Applications, chapter 8 ('Unreliable Clocks')	The §1 catalogue and the §4 warning , with real incident examples.
Kulkarni et al. — Logical Physical Clocks (HLC) (free)	The §3 default, from the paper that introduced it.
Corbett et al. — Spanner: Google's Globally-Distributed Database (free)	TrueTime and commit wait — the §1 callout, as shipped.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Measure clock drift** between two machines over an hour, with NTP disabled.
- 2 Write code that subtracts two wall-clock readings and **make it produce a negative duration** by stepping the clock.
- 3 Implement Lamport clocks and verify the guarantee empirically.

- 4 **Find two concurrent events whose Lamport timestamps are ordered** and explain why that is not a bug.
- 5 Implement vector clocks and build a tool that reports, for any two events, whether one happened before the other or they are concurrent.
- 6 Measure the bandwidth cost of vector clocks as the node count rises from 3 to 100.
- 7 **Implement a hybrid logical clock** and verify it respects causality while tracking wall time.
- 8 **Build a last-write-wins store, skew one node's clock by 200 ms, and demonstrate silent data loss.**
- 9 Replace LWW with version vectors and surface the conflict instead.
- 10 Implement client-to-server tick offset estimation and measure its stability under jitter.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give five ways a physical clock betrays you.
- 2 Why should durations use a monotonic clock?
- 3 Why is a clock reading a range, and what does TrueTime do about it?
- 4 Define happens-before and say what 'concurrent' actually means.
- 5 State what a Lamport clock guarantees and what it does not.
- 6 How do vector clocks detect concurrency, and what do they cost?
- 7 Compare six clock schemes.
- 8 Why is last-write-wins dangerous, and what are the alternatives?
- 9 How does a game define time, and why does that enable lag compensation?